

CKAD : Séance 1, Pods

Support de synthèse : exercices et corrections

Exercice 1 : Quiz de raisonnement

Énoncé

Question 1. Un Pod reste à l'état `Pending` depuis 3 minutes. Quelle commande permet d'identifier la cause, et quelle section de la sortie faut-il analyser en priorité ?

Question 2. L'énoncé précise que le Pod doit s'exécuter une seule fois et ne jamais redémarrer, même en cas d'erreur. Quelle valeur de `restartPolicy` faut-il utiliser ?

Question 3. La commande `kubectl apply -f pod.yaml` retourne `pod/my-pod created`. Cette confirmation est-elle suffisante pour valider la réponse à l'examen ?

Question 4. Quelle est la différence entre `kubectl create` et `kubectl apply` ? Laquelle privilégier à l'examen ?

Correction

Question 1.

```
kubectl describe pod <name> -n <ns>
```

Examiner la section **Events** en fin de sortie. Causes fréquentes : ressources insuffisantes sur les nodes, taint non tolérée, image impossible à puller (`ImagePullBackOff`).

Question 2.

```
restartPolicy: Never
```

`OnFailure` n'est pas adapté ici : cette valeur redémarre le container si le code de sortie est différent de 0, ce qui contredit l'énoncé. `Never` ne redémarre jamais, quel que soit le résultat.

Question 3.

Non. Le message `created` confirme uniquement que l'API a accepté le manifeste, pas que le Pod est opérationnel. Il convient de toujours vérifier l'état réel :

```
kubectl get pod my-pod -n <ns>
# Statut attendu : Running ou Completed
```

Question 4.

Commande	Comportement
<code>kubectl create</code>	Échoue si la ressource existe déjà
<code>kubectl apply</code>	Crée ou met à jour de façon idempotente

À l'examen, `apply` est à privilégier systématiquement : une nouvelle soumission par erreur n'entraîne pas d'échec.

Exercice 2 : Diagnostic et commandes kubectl

Énoncé

Cluster `k8s-staging`. Le Pod `api-pod` (namespace `backend`) est à l'état `CrashLoopBackOff`. Sans modifier la ressource, identifier la cause du crash et relever le code de sortie du dernier redémarrage.

Correction

Étape 1. Changement de contexte, à effectuer systématiquement avant toute opération :

```
kubectl config use-context k8s-staging
```

Étape 2. Démarche de diagnostic :

```
kubectl get pod api-pod -n backend
kubectl get pod api-pod -n backend -o wide      # identifier le node
kubectl describe pod api-pod -n backend         # section Events + Last State
kubectl logs api-pod -n backend
kubectl logs api-pod -n backend --previous      # logs précédant le crash
```

Étape 3. Inspection complémentaire :

```
kubectl get pod api-pod -n backend -o yaml      # manifeste complet tel que stocké
kubectl exec -it api-pod -n backend -- sh       # accès au container
kubectl explain pod.spec.containers             # documentation intégrée
```

Étape 4. Pod de diagnostic temporaire :

```
kubectl run debug --image=busybox \  
  --restart=Never --rm -it \  
  -n backend -- sh  
# --rm : suppression automatique à la sortie
```

Point d'attention : `kubectl logs` sans l'option `--previous` ne renvoie aucun résultat si le container a déjà redémarré après son crash.

Exercice 3 : Rédaction d'un manifeste Pod

Énoncé

Cluster `k8s-prod`. Créer un Pod `nginx` dans le namespace `monitoring`, avec les labels `app=nginx` et `env=prod`. Le Pod doit redémarrer automatiquement en cas d'arrêt.

Correction

Changement de contexte et vérification du namespace :

```
kubectl config use-context k8s-prod  
kubectl get ns monitoring  
kubectl create ns monitoring # si le namespace n'existe pas
```

Manifeste :

```
apiVersion: v1  
kind: Pod  
metadata:  
  name: nginx-pod  
  namespace: monitoring  
  labels:  
    app: nginx  
    env: prod  
spec:  
  restartPolicy: Always  
  containers:  
  - name: nginx  
    image: nginx:1.25
```

Vérification post-déploiement, étape obligatoire :

```
kubectl apply -f pod.yaml
kubectl get pod nginx-pod -n monitoring          # statut Running attendu
kubectl get pod nginx-pod -n monitoring --show-labels # contrôle des labels
```

Points de vigilance :

- `apiVersion: v1` pour un Pod, non `apps/v1` (réservé aux Deployments)
- Le champ `namespace` se positionne dans `metadata`, non dans `spec`
- `containers` est une liste : le tiret `-` est obligatoire devant chaque élément

Exercice 4 : Génération rapide via dry-run

Énoncé

Cluster `k8s-dev`. Créer en moins de 2 minutes un Pod `cache-pod` (image `redis:7`, namespace `infra`, label `tier=cache`), sans rédiger le manifeste manuellement.

Correction

```
kubectl config use-context k8s-dev

kubectl run cache-pod --image=redis:7 \
  -n infra --labels="tier=cache" \
  --dry-run=client -o yaml > cache-pod.yaml

kubectl apply -f cache-pod.yaml
kubectl get pod cache-pod -n infra --show-labels
```

Configuration recommandée en début d'examen :

```
alias k=kubectl
export do="--dry-run=client -o yaml"

k run cache-pod --image=redis:7 -n infra --labels="tier=cache" $do > cache-
pod.yaml
```

Pour mémoire, copie de fichiers vers ou depuis un Pod :

```
kubectl cp cache-pod:/data/fichier.txt ./local.txt -n infra
```

Points de vigilance :

- `--dry-run` sans `=client` est obsolète ; toujours utiliser `--dry-run=client`

- Omettre `-o yaml > fichier.yaml` ne génère aucun fichier sur disque
- `dry-run` ne crée rien dans le cluster ; `kubectl apply` reste nécessaire

Exercice 5 : Manifeste complet

Énoncé

Cluster `k8s-prod`. L'application `worker` (image `busybox`) ne doit démarrer qu'après l'exécution réussie d'une tâche d'initialisation (image `busybox`, commande `echo "init ok"`). L'application exécute `/bin/sh -c "echo running; sleep 3600"`. Variable d'environnement requise : `APP_ENV=production`. Limite mémoire : 128Mi, limite CPU : 250m. Namespace : `workers`.

Correction

```
apiVersion: v1
kind: Pod
metadata:
  name: worker
  namespace: workers
spec:
  initContainers:
    - name: init-task
      image: busybox
      command: ["echo", "init ok"]
  containers:
    - name: worker
      image: busybox
      command: ["/bin/sh"]
      args: ["-c", "echo running; sleep 3600"]
      env:
        - name: APP_ENV
          value: "production"
      resources:
        requests:
          memory: "64Mi"
          cpu: "125m"
        limits:
          memory: "128Mi"
          cpu: "250m"
```

Correspondance `command` / `args` avec la syntaxe Dockerfile :

Dockerfile	YAML Kubernetes
ENTRYPOINT	command
CMD	args

Vérification :

```
kubectl apply -f worker.yaml
kubectl get pod worker -n workers
# Statut intermédiaire : Init:0/1 (tâche d'initialisation en cours)
# Statut final          : Running

kubectl describe pod worker -n workers
# Vérifier la présence de APP_ENV dans la section Environment
```

Points de vigilance :

- `initContainers` (au pluriel) se situe au même niveau que `containers` dans `spec`, non à l'intérieur
- `env` est une liste de paires `{name, value}`, non un dictionnaire `clé: valeur`
- `250m` équivaut à 0.25 vCPU ; l'omission du suffixe `m` change l'ordre de grandeur de la requête et peut empêcher le scheduling
- `limits` sans `requests` entraîne une copie automatique des valeurs de `limits` vers `requests`

Complément : modification d'un Pod en cours d'exécution

Énoncé

Le Pod `nginx-pod` (namespace `monitoring`) utilise l'image `nginx:1.25`. Faire évoluer cette version vers `nginx:1.26` sans suppression manuelle du Pod.

Correction

Option 1, `edit` :

```
kubectl edit pod nginx-pod -n monitoring
# Modification de l'image dans l'éditeur, puis sauvegarde
```

Option 2, `patch` :

```
kubectl patch pod nginx-pod -n monitoring \
-p '{"spec":{"containers":[{"name":"nginx","image":"nginx:1.26"}]}}'
```

Option 3, `replace --force`, recommandée pour les champs immutables :

```
kubectl get pod nginx-pod -n monitoring -o yaml > pod.yaml
# Modification de l'image dans pod.yaml
kubectl replace --force -f pod.yaml
```

Point d'attention : `kubectl edit` refuse fréquemment la modification de champs immutables tels que l'image. Dans ce cas, `replace --force` constitue l'alternative fiable.

Complément : Pod multi-container (pattern sidecar)

Énoncé

Créer un Pod `app-with-sidecar` avec deux containers : `app` (busybox, commande `sleep 3600`) et un sidecar `logger` (busybox, commande `sh -c "while true; do date; sleep 5; done"`).

Correction

```
apiVersion: v1
kind: Pod
metadata:
  name: app-with-sidecar
spec:
  containers:
    - name: app
      image: busybox
      command: ["sleep", "3600"]
    - name: logger
      image: busybox
      command: ["sh", "-c", "while true; do date; sleep 5; done"]
```

Inspection de chaque container :

```
kubectl logs app-with-sidecar -c logger      # le paramètre -c est obligatoire
kubectl exec -it app-with-sidecar -c app -- sh
```

Vérification :

```
kubectl get pod app-with-sidecar
# La colonne READY doit afficher 2/2
```

Point d'attention : sur un Pod multi-container, `kubectl logs` sans l'option `-c <nom>` retourne une erreur.

Synthèse : points clés de la séance

1. Privilégier systématiquement `apply` plutôt que `create`
2. Vérifier l'état réel après chaque déploiement : `created` ne garantit pas `Running`
3. Préciser le namespace (`-n`) sur l'ensemble des commandes
4. Utiliser `--dry-run=client -o yaml` pour générer un manifeste sans le rédiger manuellement
5. `kubectl explain` constitue la référence documentaire pendant l'examen, en l'absence d'accès internet

Séance 1, Pods. Document à conserver pour préparation à la séance suivante.